Actividad práctica — Contenerización con Docker

Introducción

En esta actividad se desarrolló una aplicación mínima utilizando Docker, con el propósito de comprender el proceso de construcción de imágenes, ejecución de contenedores, uso de variables de entorno y persistencia básica mediante volúmenes.

La práctica consistió en crear una aplicación sencilla en Python, construir su imagen Docker y ejecutarla dentro de un contenedor accesible desde el navegador. Además, se realizaron pruebas de ejecución, inspección y limpieza del entorno.

Objetivo

Construir una imagen Docker funcional para una aplicación mínima y ejecutarla mapeando puertos, variables de entorno y un volumen de datos.

Requisitos previos

Para realizar la actividad se utilizaron los siguientes recursos:

- Docker Desktop instalado
- Visual Studio Code
- Terminal de PowerShell
- Navegador web

Estructura del proyecto

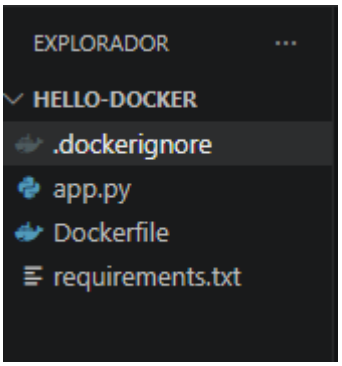
Se creó una carpeta llamada:

hello-docker

Dentro de esta carpeta se agregaron los siguientes archivos:

```
hello-docker/
│
├── app.py
├── requirements.txt
├── Dockerfile
└── .dockerignore
```

Esta estructura corresponde exactamente a la solicitada en la guía de la actividad.



Desarrollo de la aplicación mínima

En el archivo app.py se implementó un servidor HTTP simple en Python. Esta aplicación responde a las solicitudes GET mostrando un mensaje de texto en el navegador.

Además, se utilizó una variable de entorno llamada PORT, permitiendo que el puerto de ejecución pueda configurarse sin modificar directamente el código fuente.

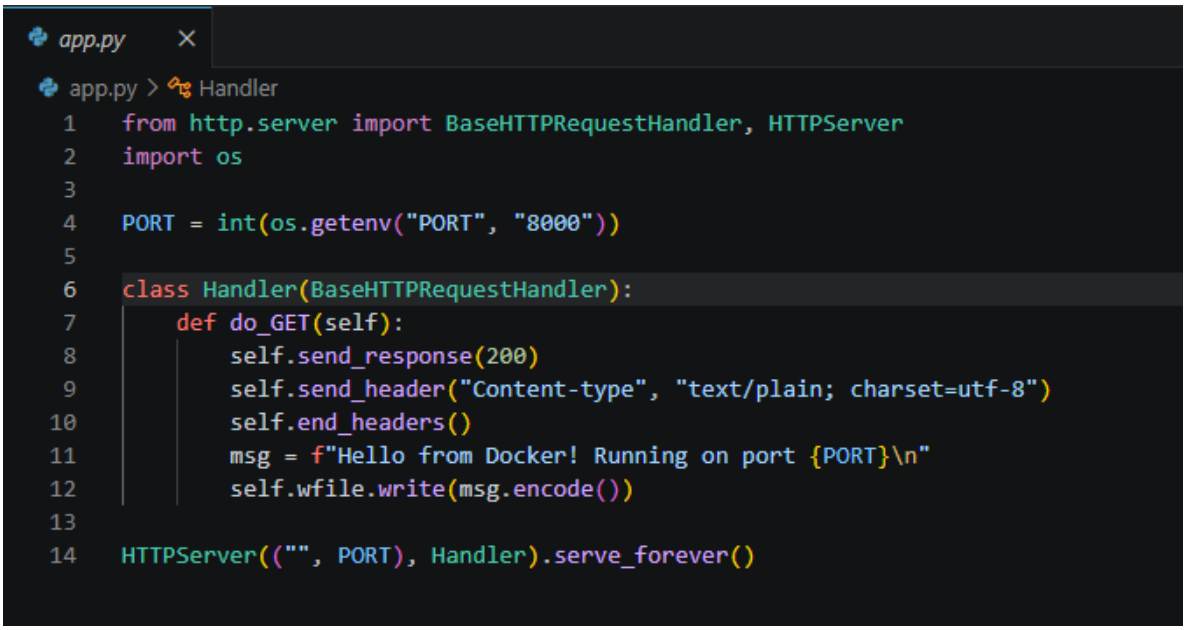
Código utilizado en app.py

```python
from http.server import BaseHTTPRequestHandler, HTTPServer
import os

PORT = int(os.getenv("PORT", "8000"))

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-type", "text/plain; charset=utf-8")
        self.end_headers()
        msg = f"Hello from Docker! Running on port {PORT}\n"
        self.wfile.write(msg.encode())

HTTPServer(("", PORT), Handler).serve_forever()
```

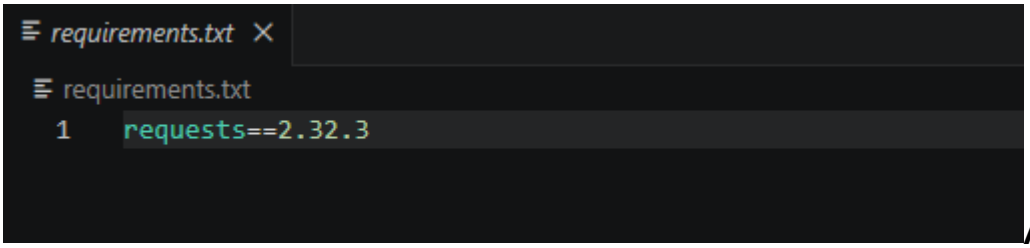


Archivo requirements.txt

Se creó el archivo requirements.txt para definir las dependencias necesarias de la aplicación. En este caso se utilizó el siguiente contenido:

requests==2.32.3

La guía también permite dejar este archivo vacío si la aplicación no requiere dependencias externas.



Archivo .dockerignore

Se creó el archivo .dockerignore con el objetivo de evitar que Docker copie archivos innecesarios al momento de construir la imagen.

Contenido utilizado

__pycache__/

*.pyc

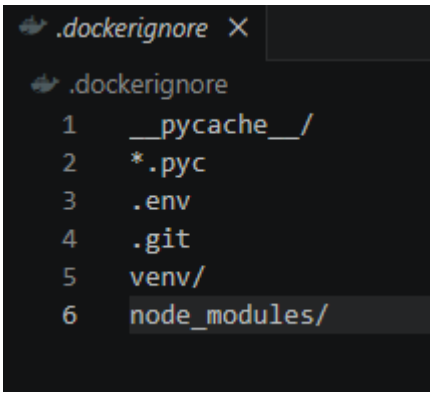
.env

.git

venv/

node_modules/

Esto ayuda a mantener la imagen más limpia y ligera.



Creación del Dockerfile

El archivo Dockerfile define las instrucciones necesarias para construir la imagen Docker de la aplicación.

Contenido utilizado

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python", "app.py"]
```

Este Dockerfile realiza las siguientes acciones:

- utiliza una imagen base de Python 3.11
- define la carpeta de trabajo /app
- copia el archivo de dependencias
- instala las dependencias
- copia el resto de los archivos del proyecto
- expone el puerto 8000
- ejecuta la aplicación principal

La estructura coincide con la guía de la actividad.

```
Run 'docker run --help' for more information
PS C:\Users\demoo\Desktop\hello-docker> Get-Content Dockerfile
FROM python:3.11-slim
WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .
```

Construcción de la imagen Docker

Una vez completados los archivos del proyecto, se construyó la imagen utilizando el siguiente comando:

docker build -t hello-docker:1.0 .

Este comando permitió generar una imagen llamada hello-docker con la etiqueta 1.0. La construcción de la imagen empaquetó la aplicación junto con todas sus dependencias dentro de un entorno aislado.

```
PS C:\Users\demoo\Desktop\hello-docker> docker build -t hello-docker:1.0 .
[+] Building 15.7s (10/10) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 212B
 => [internal] load metadata for docker.io/library/python:3.11-slim
 => [internal] load .dockerignore
 => => transferring context: 93B
 => [1/5] FROM docker.io/library/python:3.11-slim@sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bf
 => => resolve docker.io/library/python:3.11-slim@sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bf
 => => sha256:0816ee351b60c03c7a77a3f50c717891eedd927f4e6a1d59f6cce05268065606 251B / 251B
 => => sha256:d98f36ba21593a4014243f41ec9c5370abd34f4f65a5d074e41bce23f5f6a165 14.37MB / 14.37MB
 => => sha256:37ffa6577b04e98598e2c10432eb8c66cfc1af6c7698b21c6163f8334606aa14 1.29MB / 1.29MB
```

Ejecución del contenedor

Después de construir la imagen, se ejecutó el contenedor con el siguiente comando:

docker run -d -p 8000:8000 --name hello hello-docker:1.0

Este comando ejecutó el contenedor en segundo plano y realizó el mapeo del puerto 8000 del contenedor hacia el puerto 8000 del equipo local, permitiendo acceder a la aplicación desde el navegador.

```
PS C:\Users\demoo\Desktop\hello-docker> docker run -d -p 8000:8000 --name hello hello-docker:1.0
a1566ad685942f8394428637ee32e2350f9b9b0cf2adb57a7c903ffe73b6c410
PS C:\Users\demoo\Desktop\hello-docker>
```

Verificación del contenedor

Para verificar que el contenedor se encontraba en ejecución, se utilizó el siguiente comando:

docker ps

Este comando permitió comprobar que el contenedor llamado hello estaba corriendo

Verificación en el navegador

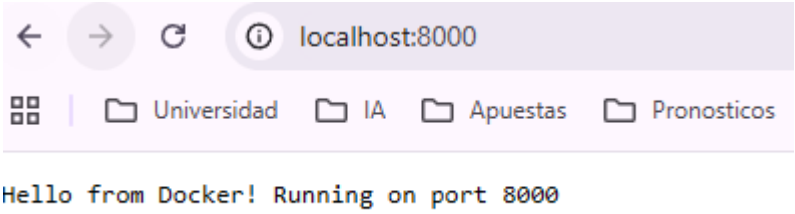
Luego se ingresó en el navegador a la siguiente dirección:

http://localhost:8000

La aplicación respondió mostrando el mensaje:

Hello from Docker! Running on port 8000

Esto confirmó que la imagen y el contenedor funcionaban correctamente.



Verificación con curl

También se realizó una validación desde la terminal usando el comando:

curl http://localhost:8000

Este comando devolvió el mismo mensaje mostrado en el navegador, sirviendo como evidencia adicional del correcto funcionamiento del contenedor. La guía solicita capturas de este paso como parte de los entregables.

Persistencia mediante bind mount

Como parte de la actividad, se implementó persistencia básica usando un volumen de tipo bind mount. Para ello se creó una carpeta llamada:

data

Después se detuvo y eliminó el contenedor anterior:

docker stop hello

docker rm hello

Luego se ejecutó nuevamente el contenedor con un volumen:

En Git Bash

docker run -d -p 8000:8000 -v $(pwd)/data:/app/data --name hello hello-docker:1.0

En PowerShell

docker run -d -p 8000:8000 -v ${PWD}/data:/app/data --name hello hello-docker:1.0

Esta configuración permite que los datos almacenados en /app/data dentro del contenedor queden persistidos en la carpeta data del equipo local.

Inspección del contenedor

Se utilizaron comandos de inspección para interactuar con el contenedor y verificar su configuración.

Acceso al contenedor

docker exec -it hello sh

Inspección detallada

docker inspect hello

Estos comandos permitieron confirmar que el contenedor estaba corriendo, revisar su configuración interna y comprobar que el volumen estaba correctamente montado.

Limpieza del entorno

Finalmente, se realizaron tareas de limpieza utilizando los siguientes comandos:

docker stop hello

docker rm hello

docker image ls

docker image prune

Estos comandos permitieron detener y eliminar el contenedor, listar las imágenes presentes y limpiar imágenes no utilizadas.

Qué aprendí sobre variables de entorno y volúmenes

Durante el desarrollo de esta actividad se comprendió que las variables de entorno permiten configurar aspectos importantes de una aplicación sin modificar directamente el código fuente. En este caso, la variable PORT permitió definir dinámicamente el puerto en el que la aplicación debía ejecutarse.

También se aprendió que los volúmenes son fundamentales para la persistencia de datos, ya que permiten conservar información fuera del contenedor. Esto es especialmente útil cuando se desea evitar la pérdida de datos al detener o eliminar contenedores.

Dificultades encontradas y cómo se resolvieron

Una de las dificultades encontradas fue que inicialmente Docker no estaba iniciado correctamente, lo que generó un error de conexión con el motor de Docker. Este problema se resolvió abriendo Docker Desktop y esperando a que el servicio estuviera completamente disponible.

Otra dificultad fue que el archivo Dockerfile estaba vacío, lo que impedía construir la imagen. La solución fue verificar que el archivo existiera con el nombre correcto y agregar el contenido adecuado antes de volver a ejecutar el comando docker build.

Finalmente, también se revisó el correcto nombre de los archivos del proyecto y el contenido del directorio para garantizar que Docker pudiera construir la imagen sin errores.

Conclusión

La actividad permitió comprender de manera práctica el proceso de contenerización de una aplicación mínima usando Docker. Se aprendió a estructurar un proyecto, crear un Dockerfile, construir imágenes, ejecutar contenedores, mapear puertos, usar variables de entorno y trabajar con persistencia básica mediante volúmenes.

Estos conocimientos son esenciales para el desarrollo de aplicaciones modernas, especialmente en entornos donde se busca portabilidad, aislamiento y facilidad de despliegue.

Comandos utilizados en la práctica

```
docker build -t hello-docker:1.0 .

docker run -d -p 8000:8000 --name hello hello-docker:1.0

docker ps

docker logs -f hello

curl http://localhost:8000

docker stop hello

docker rm hello

docker run -d -p 8000:8000 -v $(pwd)/data:/app/data --name hello hello-docker:1.0

docker exec -it hello sh

docker inspect hello

docker stop hello

docker rm hello

docker image ls

docker image prune
```

Nota para PowerShell

En PowerShell, el comando del volumen se puede ejecutar así:

```
docker run -d -p 8000:8000 -v ${PWD}/data:/app/data --name hello hello-docker:1.0
```

Si quieres, te dejo también **los 4 archivos exactos listos para copiar y pegar**: app.py, Dockerfile, .dockerignore y requirements.txt.